

Proyecto Final - Desarrollo y Arquitecturas Web

Juego de Memoria / Memotest

Link de interés: <https://www.helpfulgames.com/subjects/brain-training/memory.html>

El link anterior es únicamente de referencia visual y funcional. No se debe copiar código, estructura interna ni recursos protegidos de ese sitio. Cada grupo deberá desarrollar su propia implementación.

Descripción general

El proyecto consiste en desarrollar un juego de memoria, también conocido como memo, memotest o juego de pares.

El juego debe presentar un tablero compuesto por cartas dadas vuelta. Cada carta tendrá una imagen, ícono, texto o elemento visual oculto. El jugador deberá seleccionar dos cartas por turno intentando encontrar dos cartas iguales o relacionadas entre sí.

Si las dos cartas seleccionadas coinciden, quedarán descubiertas en el tablero. Si no coinciden, deberán volver a ocultarse luego de un breve tiempo. El objetivo del juego es encontrar todos los pares en la menor cantidad de tiempo posible, con la menor cantidad de intentos y obteniendo la mayor cantidad de puntos.

Cada grupo podrá proponer una temática para su juego. Algunos ejemplos posibles son:

- Mundial de fútbol 2026.
- Personajes de Pokémon.
- Escudos de clubes.
- Películas o series.
- Animales.
- Países y banderas.
- Videojuegos.
- Temática libre propuesta por el grupo.

La temática elegida deberá ser coherente en toda la aplicación, tanto en las imágenes como en los textos, colores, sonidos y estilos visuales.



Reglas del juego

El tablero estará formado por una cantidad par de cartas. Cada carta tendrá una pareja asociada.

Al iniciar la partida, las cartas deberán mezclarse aleatoriamente y mostrarse boca abajo.

El jugador deberá seleccionar una primera carta y luego una segunda carta.

Si ambas cartas forman un par correcto:

- Las cartas quedarán descubiertas.
- Se sumarán puntos.
- El jugador podrá continuar jugando.

Si las cartas no forman un par correcto:

- Se aplicará una penalización.
- Las cartas deberán volver a ocultarse luego de un breve tiempo.
- El jugador podrá seguir intentando encontrar los pares restantes.

La partida termina cuando el jugador encuentra todos los pares del tablero.

El juego debe permitir reiniciar la partida sin recargar la página.

Sistema de niveles

El juego deberá contar, como mínimo, con tres niveles de dificultad:

Nivel fácil

- Tablero de 4x4.
- 16 cartas en total.
- 8 pares.
- Penalización baja por error.

Nivel medio

- Tablero de 4x5 o 5x4.
- 20 cartas en total.
- 10 pares.
- Penalización media por error.



Nivel difícil

- Tablero de 6x6.
- 36 cartas en total.
- 18 pares.
- Penalización alta por error.

El jugador deberá poder seleccionar el nivel de dificultad antes de comenzar la partida.

También se podrá implementar un modo progresivo, donde el jugador comienza en nivel fácil y, al ganar, avanza automáticamente al siguiente nivel aumentando la cantidad de cartas, la dificultad y/o la penalización.

Sistema de puntaje

El juego deberá tener un sistema de puntos claro y visible para el jugador.

El grupo podrá proponer su propia fórmula de puntaje, pero deberá estar explicada en el README del proyecto.

Como referencia, se puede utilizar el siguiente criterio:

- Cada par encontrado suma puntos.
- Cada error resta puntos.
- El tiempo utilizado puede disminuir el puntaje final.
- Completar niveles más difíciles otorga mayor puntaje.
- El puntaje nunca debería quedar en valores incoherentes o imposibles de interpretar.

Ejemplo de fórmula posible:

- Par correcto: +100 puntos.
- Error en nivel fácil: -10 puntos.
- Error en nivel medio: -20 puntos.
- Error en nivel difícil: -30 puntos.
- Bonus por finalizar la partida: +300 puntos.
- Penalización por tiempo: -1 punto por cada segundo transcurrido.

La fórmula puede ser diferente, pero debe estar implementada de forma consistente y documentada.

Requerimientos Obligatorios



Los siguientes requerimientos son obligatorios y representan las condiciones mínimas para aprobar el trabajo.

El cumplimiento correcto de estos puntos corresponde a una nota mínima de aprobación, siempre que el proyecto funcione correctamente y respete las buenas prácticas solicitadas.

Código y estructura general

- Código prolijo y estricto utilizando HTML5, CSS3 y JavaScript ES5.
- No utilizar sintaxis de ES6 o superior.
- No utilizar frameworks ni librerías externas para resolver la lógica principal del juego.
- Consistencia en comentarios, commits, nombres de archivos, nombres de carpetas y estilos de código.
- Responsividad y estética del juego y de la web utilizando Flexbox.
- No deberá existir código bloqueante como `alert`, `prompt` o `confirm`.
- Utilizar modales o mensajes visuales dentro de la interfaz en lugar de ventanas bloqueantes del navegador.
- El proyecto deberá estar publicado en Github Pages.
- El repositorio deberá incluir un README claro, completo y bien formateado.

Inicio de partida

- El jugador deberá ingresar su nombre antes de comenzar la partida.
- El nombre deberá validarse con JavaScript.
- El nombre deberá tener como mínimo 3 caracteres.
- No se deberá permitir comenzar la partida si el nombre no es válido.
- El jugador deberá seleccionar un nivel de dificultad antes de comenzar.
- Se deberá mostrar un mensaje de error claro cuando los datos ingresados no sean válidos.

Tablero y cartas

- El tablero deberá generarse dinámicamente con JavaScript.
- Las cartas deberán mezclarse aleatoriamente al iniciar cada partida.
- El tablero deberá respetar la cantidad de cartas correspondiente al nivel seleccionado.
- Cada carta deberá tener una pareja.
- No debe ser posible seleccionar más de dos cartas en un mismo turno.
- No debe ser posible volver a seleccionar una carta que ya fue emparejada correctamente.
- No debe ser posible seleccionar dos veces la misma carta en el mismo turno.
- El tablero deberá visualizarse correctamente en desktop, tablet y mobile.
- Todas las cartas deberán tener una estética coherente con la temática elegida.



Mecánica del juego

- El jugador deberá poder hacer click sobre una carta para revelarla.
- Al seleccionar dos cartas, el juego deberá verificar si forman un par correcto.
- Si las cartas coinciden, deberán permanecer descubiertas.
- Si las cartas no coinciden, deberán volver a ocultarse luego de un breve intervalo.
- Mientras dos cartas incorrectas están visibles esperando ocultarse, el jugador no deberá poder seleccionar nuevas cartas.
- El juego deberá detectar automáticamente cuando todos los pares fueron encontrados.
- Al finalizar la partida, se deberá mostrar un mensaje indicando que el jugador ganó.
- El mensaje final deberá mostrar, como mínimo:
 - Nombre del jugador.
 - Nivel jugado.
 - Cantidad de intentos.
 - Cantidad de errores.
 - Tiempo total.
 - Puntaje final.

Intentos, errores y penalizaciones

- Mostrar la cantidad de intentos realizados.
- Mostrar la cantidad de pares encontrados.
- Mostrar la cantidad de errores cometidos.
- Mostrar el puntaje actual durante la partida.
- Aplicar penalizaciones por errores.
- Aplicar mayor penalización según el nivel de dificultad.
- El puntaje deberá actualizarse visualmente durante el juego.

Temporizador

- Mostrar un temporizador visible durante la partida.
- El temporizador deberá iniciar cuando el jugador revele la primera carta.
- El temporizador deberá detenerse cuando el jugador finalice la partida.
- Al reiniciar la partida, el temporizador deberá volver a cero.
- El tiempo total deberá formar parte del resultado final.

Reinicio de partida

- Agregar una opción para reiniciar la partida sin recargar la página.
- Al reiniciar, se deberá limpiar el tablero anterior.
- Al reiniciar, se deberán reiniciar intentos, errores, pares encontrados, puntaje y temporizador.
- Al reiniciar, las cartas deberán volver a mezclarse aleatoriamente.



- El jugador deberá poder iniciar una nueva partida conservando o modificando el nombre y el nivel.

Validaciones y manejo de errores

- Validar todos los inputs usando JavaScript.
- No utilizar validaciones HTML como única forma de validación.
- Mostrar mensajes de error claros y visibles.
- Evitar estados inválidos del juego.
- No permitir que el jugador interactúe con el tablero antes de iniciar la partida.
- No permitir que el jugador modifique el nivel de dificultad en medio de una partida activa, salvo que esto reinicie correctamente el juego.
- Controlar que siempre exista una cantidad par de cartas.
- Controlar que la cantidad de imágenes o elementos disponibles sea suficiente para generar los pares del nivel seleccionado.

Página de contacto

- Crear una página de Contacto.
- El formulario de contacto deberá permitir ingresar:
 - Nombre.
 - Mail.
 - Mensaje.
- Al enviar el formulario correctamente, se deberá abrir la herramienta de envío de emails predeterminada del sistema operativo.
- Las validaciones del formulario deberán realizarse con JavaScript.
- No se deben utilizar únicamente validaciones HTML.
- El nombre deberá ser alfanumérico.
- El mail deberá tener un formato válido.
- El mensaje deberá tener más de 5 caracteres.
- Si el formulario tiene errores, deberán mostrarse mensajes claros sin utilizar `alert`.

Navegación y links

- La aplicación deberá tener navegación entre la página principal del juego y la página de contacto.
- Agregar un link al repositorio de Github donde se alojó el código del juego.
- El link a Github deberá abrirse en una nueva pestaña.
- Agregar un link a la versión publicada en Github Pages.
- La navegación deberá ser clara y funcional en desktop y mobile.



Requerimientos Deseados

Los siguientes requerimientos son opcionales y permiten mejorar la nota final.

Si el proyecto cumple correctamente todos los requerimientos obligatorios, funciona sin errores, respeta las buenas prácticas y además incorpora todos o la mayoría de los requerimientos deseados de forma prolija, podrá aspirar a una nota superior, hasta llegar a 10.

Ranking con LocalStorage

- Guardar los resultados de cada partida usando LocalStorage.
- Cada resultado deberá guardar:
 - Nombre del jugador.
 - Puntaje final.
 - Nivel jugado.
 - Cantidad de intentos.
 - Cantidad de errores.
 - Fecha y hora.
 - Duración de la partida.
- Agregar un botón para mostrar un modal con el ranking de partidas.
- El ranking deberá mostrarse ordenado por puntaje de forma predeterminada.
- Agregar opción para ordenar el ranking por:
 - Puntaje.
 - Fecha.
 - Duración.
 - Nivel.
- Agregar opción para borrar el historial del ranking.
- Confirmar el borrado del ranking usando un modal propio, no **confirm**.

Modo progresivo

- Agregar un modo progresivo donde el jugador avance de nivel al completar cada tablero.
- El juego puede comenzar en nivel fácil y avanzar a medio y difícil.
- El puntaje deberá acumularse entre niveles.
- Al pasar de nivel, se deberá mostrar un mensaje intermedio.
- El jugador deberá poder continuar al siguiente nivel sin recargar la página.
- Si el jugador completa todos los niveles, se deberá mostrar un resultado final general.

Sistema avanzado de puntaje

- Agregar bonus por racha de aciertos consecutivos.
- Agregar penalización progresiva por errores consecutivos.
- Agregar bonus por terminar rápido.



- Agregar bonus por completar el juego con pocos intentos.
- Mostrar una explicación clara del cálculo del puntaje final.
- Documentar la fórmula de puntaje en el README.

Accesibilidad y experiencia de usuario

- Agregar modo oscuro y modo claro.
- Permitir cambiar entre ambos modos desde la interfaz.
- Recordar la preferencia del usuario usando LocalStorage.
- Agregar estados visuales para cartas seleccionadas, correctas e incorrectas.
- Agregar textos alternativos en imágenes cuando corresponda.
- Usar colores con buen contraste.
- Evitar depender únicamente del color para comunicar un estado.

Sonidos

- Agregar sonidos libres de derechos.
 - Ejemplos:
 - Sonido al seleccionar carta.
 - Sonido al encontrar un par.
 - Sonido al cometer un error.
 - Sonido al ganar la partida.
 - Agregar opción para activar o desactivar sonidos.
-

Aclaración sobre la nota

Los requerimientos obligatorios son básicos para aprobar el trabajo.

Cumplir únicamente con los requerimientos obligatorios, de forma correcta y funcional, permite alcanzar la nota mínima de aprobación.

Los requerimientos deseados son opcionales y suman a la nota final.

Para alcanzar una nota alta, no alcanza con agregar funcionalidades. Las funcionalidades deben estar bien implementadas, sin errores, integradas correctamente a la interfaz, documentadas y desarrolladas respetando las buenas prácticas de HTML, CSS, JavaScript, Git y Github.

Un proyecto con muchas funcionalidades pero desprolijo, inconsistente o con errores graves podrá tener menor nota que un proyecto más simple pero sólido, ordenado y correctamente implementado.



Condiciones mínimas para una correcta entrega

Tipo de nomenclaturas

Los alumnos deberán conocer y aplicar correctamente los distintos estilos de nomenclatura:

- Pascal Case: `SomeName`
- Camel Case: `someName`
- Snake Case: `some_name`
- Kebab Case: `some-name`

El proyecto deberá mantener un criterio consistente. No se deberán mezclar estilos de nombres sin justificación.

Git y Github

- El proyecto deberá estar alojado en un repositorio de Github.
- El repositorio deberá estar publicado usando Github Pages.
- Los commits deben contar el paso a paso que se realizó.
- Si se leen los mensajes de los commits, se debe poder entender cómo se construyó el proyecto.
- No hay una cantidad exacta de commits correcta, pero no tiene sentido tener muy pocos commits.
- No sirven mensajes de commit como:
 - `Fixes`
 - `Progress`
 - `Changes`
 - `Final`
 - `Entrega`
- Los mensajes de commit deben reflejar el cambio realizado en términos del progreso del proyecto.
- El mensaje del commit debe ser corto, claro y descriptivo.
- Ejemplos de buenos commits:
 - `Crear estructura inicial del proyecto`
 - `Agregar validación del nombre del jugador`
 - `Generar tablero dinámico según dificultad`
 - `Implementar comparación de cartas seleccionadas`
 - `Agregar temporizador de partida`
 - `Guardar resultados en LocalStorage`
 - `Crear modal de ranking`



- Corregir estilos responsive del tablero

Archivos y carpetas

- El repositorio debe tener un README claro, con la información necesaria del proyecto y bien formateado.
- El README deberá incluir:
 - Nombre del proyecto.
 - Descripción del juego.
 - Temática elegida.
 - Reglas del juego.
 - Sistema de puntaje.
 - Funcionalidades implementadas.
 - Link a Github Pages.
 - Integrantes del grupo, si corresponde.
- Los archivos de imágenes, CSS y JavaScript deben estar guardados en carpetas correspondientes.
- Se recomienda una estructura similar a:

```
/assets
  /images
  /sounds
/css
  reset.css
  styles.css
/js
  main.js
  game.js
  storage.js
  validations.js
/pages
  contact.html
index.html
README.md
.gitignore
```

- Los nombres de archivos y carpetas deben mantener un mismo estilo de nombre.
- Se recomienda utilizar Kebab Case para archivos y carpetas.
- Utilizar un archivo `.gitignore` es una buena práctica para evitar subir archivos ocultos del sistema operativo o del editor.
- Ejemplos de archivos a ignorar:
 - `.vscode`
 - `.DS_Store`
 - `.thumbnails`
 - Archivos temporales del sistema operativo.



HTML

- Siempre definir `DOCTYPE`.
- Siempre definir `charset`.
- Siempre definir `viewport`.
- Siempre definir un título claro para cada página.
- Siempre agregar un archivo CSS para normalizar estilos cross browser, por ejemplo `reset.css`.
- Nunca usar elementos `<br>` para separar líneas. Eso debe resolverse desde CSS.
- Las referencias a JavaScript pueden agregarse al final del elemento `body` o al final del elemento `head`, manejando correctamente la carga del DOM.
- Nunca usar estilos en línea.
- Nunca usar JavaScript en línea.
- No utilizar atributos como `onclick`, `onchange` o similares en el HTML.
- Los eventos deben manejarse desde JavaScript con `addEventListener`.
- Los nombres de ids y clases deben mantener consistencia.
- Se debe usar una estructura semántica correcta.
- Usar etiquetas como `header`, `nav`, `main`, `section`, `article`, `footer`, cuando corresponda.
- La indentación debe ser perfecta.
- No mezclar espacios con tabs.
- Revisar cómo queda el archivo indentado en Github.
- Se evalúa el código en Github, no el que se ve en el IDE de cada alumno.
- No dejar líneas en blanco innecesarias.
- No dejar comentarios innecesarios.
- No dejar código comentado sin uso.

CSS

- Utilizar Flexbox.
- No utilizar Grid.
- No utilizar Float para maquetar.
- Utilizar siempre el mismo estilo de color:
 - Hexadecimal.
 - RGB.
 - HSL.
 - Palabras clave.
- No mezclar estilos de color sin criterio.
- Utilizar unidades de medida de forma consistente.



- Se pueden combinar unidades como `px`, `rem`, `%` o `vw`, pero debe existir criterio en su uso.
- En un selector compuesto, cuando tiene un selector de ID, todo lo que está antes del `#` no es tomado en cuenta y debería borrarse.
- Las reglas deben estar ordenadas por selectores.
- Se recomienda ordenar de la siguiente manera:
 - Selectores de elementos.
 - Pseudoelementos.
 - Selectores de clase.
 - Selectores de id.
 - Selectores compuestos.
 - Media queries.
- Los media queries deben ir siempre al final del archivo.
- Ser consciente de las reglas y propiedades que se pisan usando media queries.
- Ordenar las propiedades dentro de las reglas.
- Se recomienda el siguiente orden:
 - `display`
 - `position`
 - Propiedades de flex.
 - Box model.
 - Background.
 - Texto.
 - Fuentes.
 - Animaciones y transiciones.
 - Resto de propiedades.
- La indentación dentro de las reglas y dentro de los media queries debe ser perfecta.
- No mezclar espacios con tabs.
- Revisar cómo queda el archivo indentado en Github.
- Se evalúa el código en Github, no el que se ve en el IDE de cada alumno.
- No dejar líneas en blanco innecesarias.
- No dejar comentarios innecesarios.
- No dejar código CSS sin uso.

JavaScript

- Siempre usar código estricto agregando `'use strict';` al principio de cada archivo JavaScript.
- No usar `let`.
- No usar `const`.
- No usar arrow functions.
- No usar template literals.



- No usar módulos de ES6.
- No usar ningún otro tipo de sintaxis de ES6 o superior.
- No mezclar comillas dobles y comillas simples.
- Elegir un estilo y respetarlo en todo el proyecto.
- Si se usa `;` al final de una sentencia, usarlo en todas.
- Si se crean funciones anónimas y se guardan en variables, hacerlo siempre con el mismo criterio.
- De lo contrario, usar funciones nominadas.
- Para las operaciones de comparación por igualdad usar siempre `===` y `!==`.
- La sintaxis del `if`, `else` y de los bucles `for` debe ser siempre igual.
- Declarar todas las variables globales al principio del archivo.
- Declarar las variables locales al principio de cada función.
- Manejar todos los eventos desde JavaScript con `addEventListener`.
- Evitar crear funciones anónimas directamente como callbacks.
- Es mejor extraer el código de la función fuera del callback y usar una función nominada.
- Ordenar las funciones según su aparición lógica.
- Si una función X llama a una función Y, entonces Y debería estar declarada antes que X.
- Es una buena práctica separar el JavaScript en archivos más pequeños.
- Ejemplo:
 - Archivo para inicialización.
 - Archivo para lógica del juego.
 - Archivo para validaciones.
 - Archivo para manejo de LocalStorage.
 - Archivo para eventos.
- Evitar inyectar grandes bloques de HTML desde JavaScript.
- Siempre que sea posible, tener el HTML escrito en el archivo correspondiente y mostrarlo u ocultarlo modificando clases desde JavaScript.
- La generación dinámica del tablero sí deberá realizarse con JavaScript.
- La lógica del juego deberá estar separada de la manipulación visual siempre que sea posible.
- La indentación debe ser perfecta.
- No mezclar espacios con tabs.
- Revisar cómo queda el archivo indentado en Github.
- Se evalúa el código en Github, no el que se ve en el IDE de cada alumno.
- No dejar líneas en blanco innecesarias.
- No dejar comentarios innecesarios.
- No dejar código comentado sin uso.
- No dejar `console.log` innecesarios en la entrega final.

General

- No mezclar código en inglés y en español.



- Escribir todo el proyecto en español o todo el proyecto en inglés.
 - Mantener consistencia en:
 - Nombres de variables.
 - Nombres de funciones.
 - Nombres de archivos.
 - Nombres de carpetas.
 - Clases.
 - IDs.
 - Comentarios.
 - Commits.
 - La interfaz debe ser clara para el usuario.
 - El juego debe poder utilizarse sin explicación externa.
 - Los mensajes de error deben ser comprensibles.
 - El diseño debe acompañar la temática elegida.
 - No se evaluará solamente que el juego funcione, sino también la calidad del código, la organización del proyecto, la prolijidad visual, la publicación en Github Pages y el uso correcto de Git.
-

Entrega

La entrega deberá incluir:

- Link al repositorio de Github.
- Link a la aplicación publicada en Github Pages.
- README completo.
- Código fuente completo.
- Recursos utilizados correctamente organizados.
- Commits claros y descriptivos.
- Juego funcional.
- Página de contacto funcional.
- Validaciones implementadas con JavaScript.
- Diseño responsive.
- Cumplimiento de buenas prácticas de HTML, CSS y JavaScript.

No se aceptarán entregas que dependan de archivos locales de la computadora del alumno o que no puedan ejecutarse desde Github Pages.

Entrega previa obligatoria y defensa oral

Para poder presentarse en la mesa de examen final, los alumnos deberán enviar con una semana de anticipación:



- Link al repositorio de Github.
- Link a la aplicación publicada en Github Pages.

Ambos links deberán ser enviados por correo electrónico a:

- NicolasGabrielAndres.KinderRoberts@uai.edu.ar
- Dario.Maranes@uai.edu.ar

El envío deberá realizarse al docente que corresponda según la comisión o indicación de la cátedra.

Esta entrega previa es obligatoria y tiene como objetivo permitir una primera instancia de corrección del código, revisión de buenas prácticas y prueba de la funcionalidad general del proyecto.

Luego de esta revisión inicial, la cátedra informará si el proyecto se encuentra en condiciones de pasar a la instancia de defensa oral en la mesa de examen final.

La entrega técnica del proyecto no garantiza por sí sola la aprobación del examen final.

Aunque el desarrollo esté completo, funcione correctamente y cumpla con todos los requerimientos técnicos solicitados, la aprobación final quedará sujeta a la defensa oral del trabajo ante el tribunal de la mesa de examen final.

El tribunal podrá realizar preguntas sobre cualquier parte del código, la funcionalidad, la estructura del proyecto, el uso de Git/Github y las decisiones tomadas durante el desarrollo.

La falta de conocimiento sobre el propio desarrollo, la imposibilidad de explicar el código entregado o la falta de defensa adecuada del proyecto podrá afectar la nota final o impedir la aprobación, incluso si la aplicación funciona correctamente.

